

Aim: Basics of Dart Language and Basic Flutter App

Theory:

Part 1 — Basics of Dart Language

1. What is Dart?

Dart is an open-source, object-oriented, strongly typed programming language developed by Google. It is the primary language used to build Flutter applications.

It supports both AOT (Ahead-of-Time) compilation for fast execution and JIT (Just-in-Time) compilation for quick development cycles with hot reload.

2. History of Dart

Dart was unveiled by Google at the GOTO conference in October 2011. It was originally designed as a replacement for JavaScript in web development. Over time, it evolved into the core language of the Flutter framework, launched in 2018, which made Dart widely popular for cross-platform mobile development.

3. Features of Dart

Dart is a strongly typed language, meaning every variable has a specific type and type errors are caught at compile time rather than at runtime. It supports type inference, so the compiler can automatically determine the type of a variable without the developer explicitly writing it.

4. Variables and Data Types

The main data types in Dart are `int` for whole numbers, `double` for decimal numbers, `String` for text, `bool` for true or false values, and `dynamic` for a variable that can hold any type of value. Variables can be declared using the explicit type name, using `var` where the compiler infers the type, or using `dynamic` where the type can change at runtime.

The `final` keyword is used to declare a variable whose value can be set only once at runtime, while `const` is used for values that are known and fixed at compile time.

5. Operators

Dart supports all standard arithmetic operators such as addition, subtraction, multiplication, division, integer division, and modulus. Comparison operators like `greater than`, `less than`, `equal to`, and `not equal to` are used to compare values and return a boolean result.

Logical operators such as `AND`, `OR`, and `NOT` are used to combine boolean expressions. Dart also provides null-aware operators.

6. Strings

A String in Dart is a sequence of characters enclosed in single or double quotes. Dart supports string interpolation, which means variables and expressions can be embedded directly inside a string using the dollar sign followed by the variable name, or the dollar sign with curly braces for expressions.

7. Control Flow

Control flow statements determine the order in which instructions are executed in a program. The if-else statement is used to execute a block of code only when a certain condition is true.

If the condition is false, the else block is executed instead. Multiple conditions can be chained using else if. The switch statement is used when a variable needs to be compared against multiple specific values.

8. Loops

The for loop is used when the number of iterations is known in advance. It consists of an initializer, a condition, and an increment expression.

The while loop continues executing as long as a given condition remains true, and is used when the number of iterations is not known beforehand.

The do-while loop is similar to the while loop but executes the block at least once before checking the condition. The for-in loop is used to iterate directly over the elements of a collection such as a list or set.

The forEach method is a functional approach to iterate over every element in a collection and perform an operation on each.

9. Functions

A function is defined by specifying its return type, name, parameters, and body. If a function does not return any value, its return type is written as void. Dart also supports arrow functions for single-expression functions, which use the arrow symbol instead of curly braces and a return statement. Dart functions support optional parameters. Named parameters are written inside curly braces and must be referred to by name when calling the function.

Positional optional parameters are written inside square brackets and can be omitted when calling the function. Default values can be provided for optional parameters so that they have a value even if not supplied by the caller.

10. Collections

A List is an ordered collection of elements that allows duplicate values. Elements are accessed by their index, starting from zero. Lists are similar to arrays in other languages but are dynamically sized in Dart.

A Set is an unordered collection that does not allow duplicate values. It is useful when uniqueness of elements is important and the order of elements does not matter.

A Map is a collection of key-value pairs where each key is unique. Values are accessed using their corresponding keys rather than by index. Maps are similar to dictionaries in Python or objects in JavaScript.

11. Null Safety

Null safety is a feature in Dart that prevents variables from holding a null value unless they are explicitly declared as nullable. A nullable variable is declared by adding a question mark after the type name. A non-nullable variable must always be initialized with a value and can never be null.

Null safety helps eliminate null pointer exceptions, which are one of the most common causes of runtime crashes in programs. Dart provides several null-aware operators to work safely with nullable values without causing errors.

12. Asynchronous Programming

Asynchronous programming allows a program to start a long-running task and continue executing other code without waiting for that task to finish. This is essential in applications that perform operations like reading files, making network requests, or querying databases.

In Dart, a Future represents a value that will be available at some point in the future. The `async` keyword is used to mark a function as asynchronous, and the `await` keyword is used inside an `async` function to pause execution until a Future completes and returns its value.

This makes asynchronous code easy to read and write in a style that resembles regular synchronous code.

13. Difference Between Dart and Other Languages

Dart is similar to Java and C# in its syntax and object-oriented structure but is simpler and more concise. Unlike JavaScript, Dart is strongly typed and has sound null safety built in. Compared to Python, Dart is faster in execution due to AOT compilation.

Dart's biggest advantage over most languages is its deep integration with the Flutter framework, which allows a single Dart codebase to produce applications for Android, iOS, web, and desktop platforms simultaneously.

Part 2 — Basic Flutter App

What is Flutter?

Flutter is an open-source UI toolkit developed by Google for building natively compiled applications for mobile (Android, iOS), web, and desktop from a single Dart codebase. Flutter uses widgets as the basic building blocks of the UI — everything in Flutter is a widget.

Code (main.dart):

```
import 'package:flutter/material.dart';
import
'package:firebase_core/firebase_core.dart
';
import 'firebase_options.dart';
import 'screens/crew_list_screen.dart';

void main() async {

WidgetsFlutterBinding.ensureInitialized()
;
  await Firebase.initializeApp(
    options:
DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Straw Hat Pirates Crew',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        primarySwatch: Colors.orange,
        useMaterial3: true,
        appBarTheme: const AppBarTheme(
          backgroundColor: Colors.orange,
          foregroundColor: Colors.white,
          titleTextStyle: TextStyle(
            color: Colors.white,
            fontSize: 20,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
      home: const CrewListScreen(),
    );
  }
}
```

Code (crew_list_screen.dart):

```
import 'package:flutter/material.dart';
import
'../services/firestore_service.dart';
import '../services/initial_data.dart';
import '../models/strawhat_model.dart';
import 'add_edit_crew_screen.dart';

class CrewListScreen extends
StatefulWidget {
  const CrewListScreen({super.key});

  @override
  State<CrewListScreen> createState() =>
_CrewListScreenState();
}

class _CrewListScreenState extends
State<CrewListScreen> {
  final FirestoreService
_firestoreService = FirestoreService();
  final InitialDataService
_initialDataService =
InitialDataService();
  bool _isLoading = false;

  @override
  void initState() {
    super.initState();
    _initializeData();
  }

  Future<void> _initializeData() async {
    await
_initialDataService.insertInitialDataIfNe
eded();
  }

  Future<void> _resetCollection() async {
    setState(() => _isLoading = true);

    try {
      await
_firestoreService.deleteAllCrewMembers();

      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(
            content: Text('All crew
members deleted successfully'),
            backgroundColor:
Colors.orange,
          ),
        );
      }
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(
            content: Text('Error deleting
crew: $e'),
            backgroundColor: Colors.red,
          ),
        );
      }
    } finally {
      if (mounted) {
        setState(() => _isLoading =
false);
      }
    }

    Future<void> _addInitialCrew() async {
      setState(() => _isLoading = true);

      try {
        await
_initialDataService.insertInitialDataIfNe
eded();

        if (mounted) {
          ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(
              content: Text('Initial crew
members added successfully'),
              backgroundColor:
Colors.green,
            ),
          );
        }
      } catch (e) {
        if (mounted) {
          ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(
              content: Text('Error adding
crew: $e'),
              backgroundColor: Colors.red,
            ),
          );
        }
      } finally {
        if (mounted) {
          setState(() => _isLoading =
false);
        }
      }
    }
  }
}
```

```

    }
  } catch (e) {
    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('Error deleting
crew: $e'),
          backgroundColor: Colors.red,
        ),
      );
    }
  } finally {
    if (mounted) {
      setState(() => _isLoading =
false);
    }
  }
}

Future<void> _addInitialCrew() async {
  setState(() => _isLoading = true);

  try {
    await
_initialDataService.insertInitialDataIfNe
eded();

    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('Initial crew
members added successfully'),
          backgroundColor:
Colors.green,
        ),
      );
    }
  } catch (e) {
    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('Error adding
crew: $e'),
          backgroundColor: Colors.red,
        ),
      );
    }
  } finally {
    if (mounted) {
      setState(() => _isLoading =
false);
    }
  }
}
```

```

        setState(() => _isLoading =
false);
    }
}

void _showResetConfirmationDialog() {
  showDialog(
    context: context,
    builder: (BuildContext context) {
      return AlertDialog(
        title: const Text('Delete All
Crew Members'),
        content: const Text(
          'Are you sure you want to
delete ALL Straw Hat members? '
          'This action cannot be
undone.'
        ),
        actions: [
          TextButton(
            onPressed: () =>
Navigator.pop(context),
            child: const
Text('Cancel'),
          ),
          TextButton(
            onPressed: () {
              Navigator.pop(context);
              _resetCollection();
            },
            style:
TextButton.styleFrom(foregroundColor:
Colors.red),
            child: const Text('Delete
All'),
          ),
        ],
      );
    },
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Straw Hat
Pirates Crew'),
      actions: [
        IconButton(
          icon: const
Icon(Icons.delete_sweep),
          onPressed:
_showResetConfirmationDialog,
          tooltip: 'Delete all',
        ),

```

```

        IconButton(
          icon: const
Icon(Icons.person_add),
          onPressed: _addInitialCrew,
          tooltip: 'Add initial crew',
        ),
      ],
    ),
    body: _isLoading
      ? const Center(
        child: CircularProgressIndicator(
          valueColor:
AlwaysStoppedAnimation<Color>(Colors.oran
ge),
        ),
      )
      : _buildCrewGrid(),
    floatingActionButton:
FloatingActionButton(
      onPressed: () =>
_navigateToAddScreen(context),
      backgroundColor: Colors.orange,
      child: const Icon(Icons.add,
color: Colors.white),
    ),
  );
}

Widget _buildCrewGrid() {
  return
StreamBuilder<List<StrawHatModel>>(
    stream:
_firestoreService.getCrewMembers(),
    builder: (context, snapshot) {
      if (snapshot.hasError) {
        return Center(
          child: Column(
            mainAxisAlignment:
MainAxisAlignment.center,
            children: [
              const
Icon(Icons.error_outline, size: 48,
color: Colors.red),
              const SizedBox(height:
16),
              Text('Error:
${snapshot.error}'),
              const SizedBox(height:
16),
              ElevatedButton(
                onPressed:
_initializeData,
                child: const
Text('Retry'),
              ),
            ],
          ),
        ),
      ),
    ),
  );
}

```

```

    );
  }

  if (snapshot.connectionState ==
  ConnectionState.waiting) {
    return const Center(
      child:
CircularProgressIndicator(
      valueColor:
AlwaysStoppedAnimation<Color>(Colors.oran
ge),
    ),
  );
}

final crew = snapshot.data ?? [];

if (crew.isEmpty) {
  return Center(
    child: Column(
      mainAxisAlignment:
MainAxisAlignment.center,
      children: [
        const Icon(
          Icons.person_off,
          size: 64,
          color: Colors.grey,
        ),
        const SizedBox(height:
16),
        const Text(
          'No crew members
found',
          style:
TextStyle(fontSize: 18, color:
Colors.grey),
        ),
        const SizedBox(height:
16),
        ElevatedButton(
          onPressed:
_addInitialCrew,
          style:
ElevatedButton.styleFrom(
            backgroundColor:
Colors.orange,
          ),
          child: const Text('Load
Initial Crew'),
        ),
      ],
    ),
  );
}

return GridView.builder(
  padding: const

```

```

EdgeInsets.all(12),
    gridDelegate: const
SliverGridDelegateWithFixedCrossAxisCount
(
    crossAxisCount: 5,
    childAspectRatio: 0.6,
    crossAxisSpacing: 8,
    mainAxisSpacing: 8,
  ),
  itemCount: crew.length > 10 ?
10 : crew.length,
  itemBuilder: (context, index) {
    final member = crew[index];
    return
_buildCrewCard(member);
  },
);
};

Widget _buildCrewCard(StrawHatModel
member) {
  return Card(
    elevation: 4,
    shape: RoundedRectangleBorder(
      borderRadius:
BorderRadius.circular(12),
    ),
    child: Column(
      crossAxisAlignment:
CrossAxisAlignment.start,
      children: [
        Expanded(
          flex: 20,
          child: ClipRRect(
            borderRadius: const
BorderRadius.vertical(
              top: Radius.circular(12),
            ),
            child: Image.asset(
              member.imageAsset,
              fit: BoxFit.cover,
              width: double.infinity,
              errorBuilder: (context,
error, stackTrace) {
                return Container(
                  color:
Colors.grey[300],
                  child: Center(
                    child: Column(

```

Icons. <i>broken_image</i> ,			maxLines: 1,
	size: 40,		overflow:
Colors. <i>grey</i> [600],	color:	TextOverflow.ellipsis,	
	),		
	const		Container(
SizedBox(height: 8),			padding: const
	Text(	EdgeInsets.symmetric(	
member.name[0],		horizontal: 6,	
	style: const	vertical: 3,	
TextStyle(		),	decoration:
	fontSize:	BoxDecoration(	
24,		color:	
FontWeight. <i>bold</i> ,	fontWeight:	Colors. <i>orange</i> .shade100,	borderRadius:
		BorderRadius.circular(8),	
	),		),
	),		child: Row(
	],		mainAxisSize:
	),	MainAxisSize.min,	
	);		children: [
	},		const Icon(
	),	Icons. <i>monetization_on</i> ,	
	),		size: 12,
Expanded(			color:
flex: 4,		Colors. <i>orange</i> ,	
child: Padding(			),
padding: const			const
EdgeInsets.all(8.0),		SizedBox(width: 2),	
child: Column(			Expanded(
crossAxisAlignment:			child: Text(
CrossAxisAlignment.start,		member.formattedBounty,	
mainAxisAlignment:			style:
MainAxisAlignment.spaceBetween,		TextStyle(	
children: [			fontSize:
Text(		10,	
member.name,		FontWeight. <i>bold</i> ,	fontWeight:
style: const			color:
TextStyle(	fontWeight:	Colors. <i>orange</i> .shade900,	
FontWeight. <i>bold</i> ,			),
	fontSize: 14,		overflow:
	),	TextOverflow.ellipsis,	
maxLines: 2,			),
overflow:			],
TextOverflow.ellipsis,			),
			),
	Text(		
	member.role,	<i>buttons</i>	
	style: TextStyle(		Row(
	fontSize: 10,		mainAxisAlignment:
Colors. <i>grey</i> [600],	color:	MainAxisAlignment.end,	
	),		children: [

```

        IconButton(
            icon: const
            Icon(Icons.edit, size: 16),
            color:
            Colors.blue,
            onPressed: () =>
            _navigateToEditScreen(context, member),
            constraints:
            const BoxConstraints(),
            padding: const
            EdgeInsets.all(2),
        ),
        IconButton(
            icon: const
            Icon(Icons.delete, size: 16),
            color:
            Colors.red,
            onPressed: () =>
            _showDeleteDialog(context, member),
            constraints:
            const BoxConstraints(),
            padding: const
            EdgeInsets.all(2),
        ),
    ],
),
),
),
),
),
),
),
),
),
);
}

void _navigateToAddScreen(BuildContext
context) {
    Navigator.push(
        context,
        MaterialPageRoute(
            builder: (context) => const
            AddEditCrewScreen(),
        ),
    );
}

void _navigateToEditScreen(BuildContext
context, StrawHatModel member) {
    Navigator.push(
        context,
        MaterialPageRoute(

```

```

        builder: (context) =>
AddEditCrewScreen(member: member),
    ),
  );
}

void _showDeleteDialog(BuildContext
context, StrawHatModel member) {
  showDialog(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Delete Crew
Member'),
      content: Text('Are you sure you
want to remove ${member.name} from the
crew?'),
      actions: [
        TextButton(
          onPressed: () =>
Navigator.pop(context),
          child: const Text('Cancel'),
        ),
        TextButton(
          onPressed: () async {
            Navigator.pop(context);
            await
_firestoreService.deleteCrewMember(member
.id!);

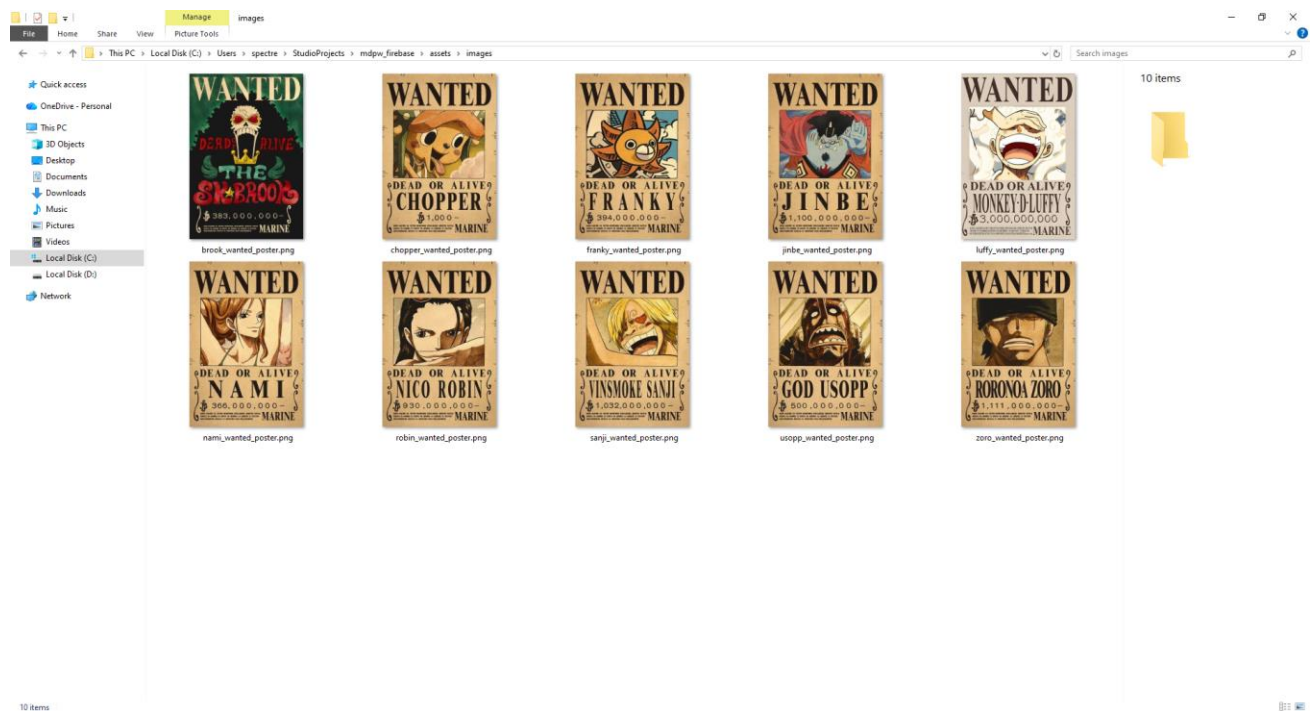
            if (mounted) {

ScaffoldMessenger.of(context).showSnackBa
r(
          Snackbar(
            content:
Text('${member.name} removed from crew'),
            backgroundColor:
Colors.red,
          ),
        ),
      ],
    ),
  );
}
}
}

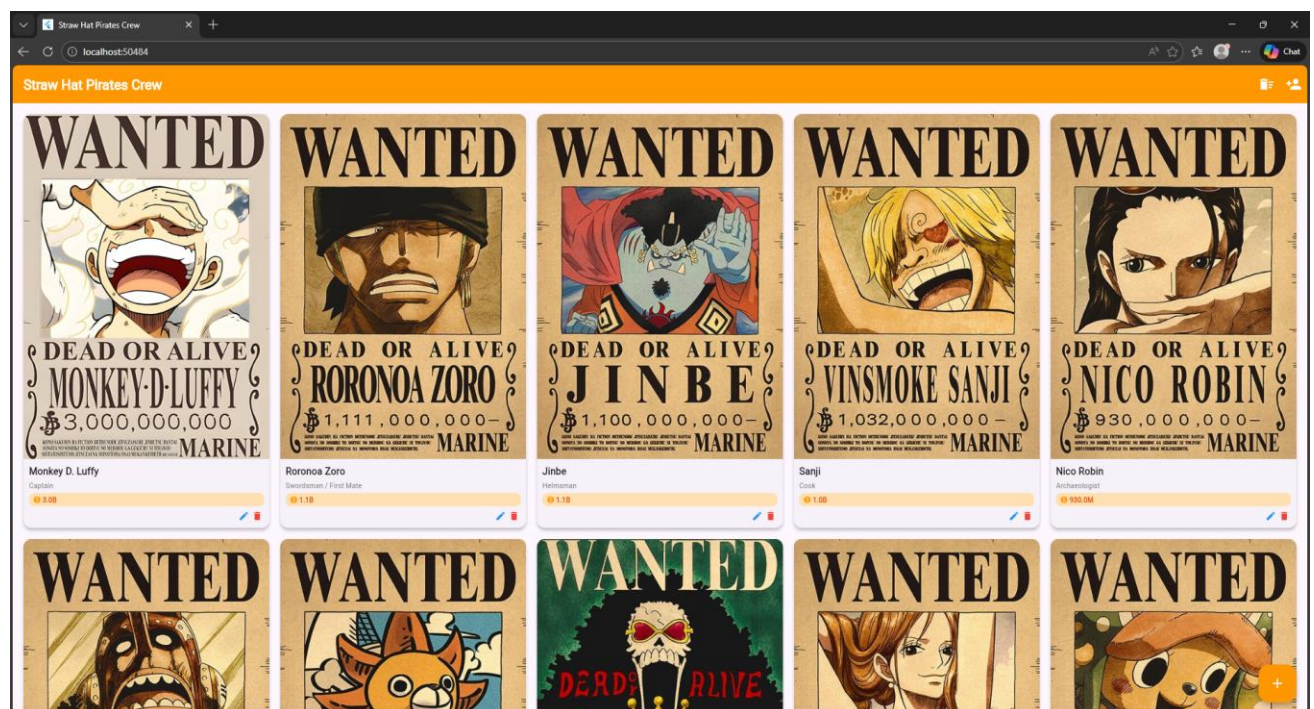
```


Output:

/assets (Wanted Posters)



Home Screen

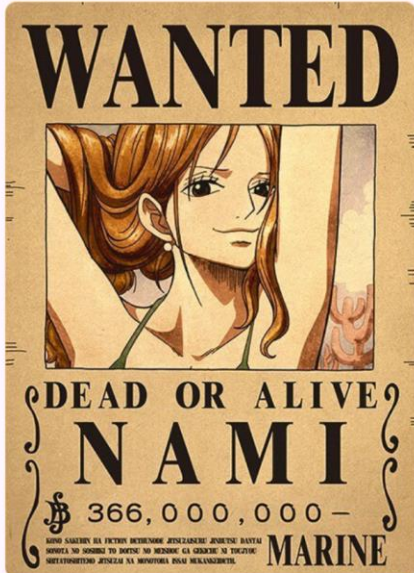


Crew Member Edit

Straw Hat Pirates Crew

localhost:50484

Edit Crew Member



WANTED

DEAD OR ALIVE

NAMI

366,000,000 -

MARINE

Name: Nami

Age: 20

Role: Navigator

Alias: Cat Burglar Nami

Bounty: 366000000

Gender: Female

Update Member

Straw Hat Pirates Crew

localhost:51486

Edit Crew Member



WANTED

DEAD OR ALIVE

MONKEY-D-LUFFY

3,000,000,000

MARINE

Name: Monkey D. Luffy

Age: 19

Role: Captain

Alias: Straw Hat Luffy

Bounty: 3000000000

Gender: Male

Update Member